

# LTMS — คู่มือการใช้งานไฟล์ OpenAPI YAML เบื้องต้น

เวอร์ชัน: 0.1 วันที่: 2 สิงหาคม 2569 สำหรับ: ทีม Backend (3 คน) และ Frontend ไฟล์ที่เกี่ยวข้อง:

- openapi/LTMS\_OpenAPI\_MVP.yaml — 92 endpoint (MVP/R01)
- openapi/LTMS\_OpenAPI\_Sprint1-2.yaml — 50 endpoint (Sprint #1/#2, R02)

## ไฟล์นี้คืออะไร

ไฟล์ `.yaml` ทั้งสองคือ สัญญาที่เครื่องอ่านได้ ระบุว่า API แต่ละ endpoint รับ-ส่งข้อมูลหน้าตาแบบไหน ไม่ใช่โค้ดที่รันเอง แต่เป็น input ให้เครื่องมืออื่นเอาไปใช้ต่อ

กฎเดียวที่ต้องจำ: ถ้าแก้ backend แล้วต้องแก้ YAML ให้ตรงกันเสมอ ไม่งั้นทุกขั้นตอนด้านล่างนี้จะให้ผลลัพธ์ที่ไม่ตรงกับความจริง

## สารบัญการใช้งานตามช่วงเวลา

| ช่วงเวลาของโปรเจกต์                 | ใช้ทำอะไร                                  | ไปที่หัวข้อ |
|-------------------------------------|--------------------------------------------|-------------|
| เริ่มโปรเจกต์ (backend ยังไม่เสร็จ) | Mock server ให้ frontend เริ่มทำงานคู่ขนาน | 1           |
| ระหว่าง dev ทุกวัน                  | เปิดดู/ทดสอบยิง API แทน Postman            | 2           |
| ทุกครั้งที่แก้ API                  | Generate TypeScript type ให้ frontend      | 3           |
| ก่อน deploy จริง                    | Contract testing เช็คว่ backend ตรงสเปค    | 4           |

## 1. สร้าง Mock Server — ใช้ตอน backend ยังไม่เสร็จ

ปัญหาที่แก้: Frontend อยากเริ่มเขียนหน้าจอแล้ว แต่ backend ยังไม่มี endpoint ให้ยิงจริง

คำสั่ง

```
npx @stoplight/prism-cli mock openapi/LTMS_OpenAPI_MVP.yaml
```

รันแล้วจะได้เซิร์ฟเวอร์ปลอมที่ `http://localhost:4010` ยิง request ไปที่นั่นได้เหมือน backend จริง จะได้ response ตามตัวอย่างที่เขียนไว้ใน YAML แทนที่

ใครใช้: Frontend ในช่วงต้นโปรเจกต์ ก่อนที่ backend คนที่ 1-3 จะเขียน endpoint นั้นเสร็จ

## 2. เปิด Swagger UI — ใช้ทุกวันระหว่างพัฒนา

ปัญหาที่แก้: ไม่อยากสลับไปเปิด Postman ทุกครั้งที่จะทดสอบยิง API

วิธีที่ 1 — ออนไลน์ ไม่ต้องติดตั้งอะไร

- เปิด <https://editor.swagger.io>
- File → Import File → เลือกไฟล์ YAML
- เห็นทุก endpoint พร้อมปุ่ม "Try it out"

วิธีที่ 2 — รันในเครื่อง

```
npx swagger-ui-watcher openapi/LTMS_OpenAPI_MVP.yaml
```

ก่อนกด Execute ต้องเช็ค 2 อย่าง

- backend รันอยู่จริงที่ `http://localhost:3000` (หรือพอร์ตที่ใช้)
- เลือก server ให้ตรง — เปิด dropdown ด้านบนของหน้า สลับจาก `https://api.ltms.example.com` (ตัวอย่างในไฟล์) เป็น `http://localhost:3000/api/v1`
- endpoint ที่ต้องล็อกอิน ให้กด **Authorize** (รูปกุญแจ) แล้ววาง token ที่ได้จาก `POST /auth/login` ก่อน

ใครใช้: ทั้งทีม backend (ทดสอบเอง) และ frontend (ดูว่า field ชื่ออะไร รับอะไรบ้าง)

## 3. Generate TypeScript Type — ทำทุกครั้งที่แก้ API

ปัญหาที่แก้: Frontend เขียน field ผิด (เช่น `sportType` แทน `sportTypeId` ) แล้วไม่รู้ตัวจนกว่าจะรันจริง

คำสั่ง

```
npx openapi-typescript openapi/LTMS_OpenAPI_MVP.yaml -o src/types/api-mvp.d.ts
npx openapi-typescript openapi/LTMS_OpenAPI_Sprint1-2.yaml -o src/types/api-sprint.d.ts
```

Frontend เอาไปใช้แบบนี้

```
import type { paths } from './types/api-mvp';

type CreateTeamBody = paths['/teams']['post']['requestBody']['content']['application/json'];

const body: CreateTeamBody = { name: 'ทีมวิสา A', sportType: 3 };
// ✕ TypeScript แดงทันที - ต้องเป็น sportTypeId ไม่ใช่ sportType
```

ท่าเมื่อไหร่: ทุกครั้งที่มีคนแก้ YAML (เพิ่ม field ใหม่, เปลี่ยนชื่อ, เพิ่ม endpoint) — แจ้งในกลุ่มทีมว่าแก้แล้ว ให้ frontend รัน command นี้ใหม่

ใครใช้: Frontend

4. Contract Testing — ทำก่อน Deploy

ปัญหาที่แก้: backend เขียนโค้ดจริงไม่ตรงกับที่ YAML สัญญาไว้ (ใส่ field, ตอบ status code ผิด) โดยไม่มีใครรู้ตัว

คำสั่ง

```
npx dredd openapi/LTMS_OpenAPI_MVP.yaml http://localhost:3000
```

อิง endpoint จริงทุกตัวเทียบกับ spec ถ้าไม่ตรงจะรายงานเป็นรายการว่า endpoint ไหนผิดตรงไหน

ท่าเมื่อไหร่: หลัง implement endpoint กลุ่มหนึ่งเสร็จ ก่อนจะรวมเข้า branch หลัก หรือก่อน deploy จริง

ใครใช้: Backend (self-check) — เป็นขั้นตอนเสริม ไม่บังคับ แต่ช่วยจับปัญหาได้เร็วกว่าไปเจอตอน frontend ใช้งานจริงแล้วพัง

ปัญหาที่พบบ่อย และวิธีแก้

| อาการ                                                      | สาเหตุ                                                                                           | วิธีแก้                                                                         |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| กด Execute แล้วขึ้น "Failed to fetch"                      | server ใน dropdown ไม่ตรงกับที่ backend รันอยู่จริง (ยังชี้ไป api.ltms.example.com ที่ไม่มีจริง) | สลับ dropdown เป็น http://localhost:3000/api/v1                                 |
| ยิงแล้วได้ 401                                             | ยังไม่ได้กด Authorize ใส่ token หรือ token หมดอายุ                                               | Execute /auth/login ก่อน คัดลอก token ไปวางที่ปุ่ม Authorize                    |
| ยิงแล้วได้ 400 จาก JSON parser (ไม่ใช่ error ที่เราออกแบบ) | พิมพ์ JSON ผิด syntax เช่นใส่ตัวอักษรในช่องที่ควรเป็นตัวเลข                                      | เช็ค raw JSON ให้ถูก โดยเฉพาะเวลาพิมพ์ตรง textarea เอง ไม่ใช่ผ่านฟอร์ม          |
| Frontend ได้ undefined ทั้งที่ type ไม่ฟ้อง error          | backend จริงส่ง field ไม่ตรงกับที่ YAML บอก (เช่น backend ส่ง full_name แต่ YAML บอก fullName )  | รัน contract testing (ข้อ 4) หรือเช็ค mapper ผ่ง backend ว่าแปลงชื่อฟิลด์ถูกไหม |
| type ที่ frontend ใช้ต่ำกว่า API จริง                      | แก้ YAML แล้วไม่ได้แจ้งให้ frontend generate type ใหม่                                           | แจ้งในทีมทุกครั้งทีแก้ YAML                                                     |

สรุปเป็นภาพเดียว

เริ่มโปรเจกต์

- (1) Mock server → frontend เริ่มเขียนคู่ขนาน

ระหว่าง dev ทุกวัน

- (2) Swagger UI → ทดสอบยิง API แทน Postman
- (3) Generate type → frontend ได้เขียนโค้ดแบบมี type ป้องกัน field ผิด

ก่อน deploy

- (4) Contract testing → เช็ค ว่า backend ตรงกับสัญญาที่เขียนไว้จริง

สิ่งที่ต้องระวังตลอดทั้งโปรเจกต์: YAML ไม่ตรวจอะไรเองอัตโนมัติ ต้องมีคนดูแลให้ตรงกับโค้ดจริงเสมอ — ถ้าแก้ backend แล้วลืมแก้ YAML ทั้ง 4 ขั้นตอนข้างต้นจะเริ่มให้ผลลัพธ์ผิดจากความจริงทันที